Scaling, Tradeoffs, and Resilience

CS6650 Midterm Mastery

Sai Karthikeyan Sura

March 2nd, 2026

HW6: Finding the Bottleneck

- **The Setup:** 100k products in a `sync.Map`, ECS Fargate (256 CPU / 512MB).
- **The Stress Test:** 20 users pushed 134.6 RPS, but CPU only hit ∼21%.
- **The Discovery:** Doubling compute to 512 CPU yielded almost no throughput gain (135.3 RPS).
- **The Takeaway:** The workload is highly compute-efficient (∼100 μs server-side). Client-observed latency (19-21ms) was dominated by Network RTT.

Test	Users	RPS	Median	p99
Baseline (256 CPU)	5	5.4	21ms	83ms
Breaking (256 CPU)	20	134.6	19ms	67ms
Breaking (512 CPU)	20	135.3	20ms	58ms

HW6: Horizontal Scaling & Auto-Recovery

- **Scaling Up:** Deployed behind an ALB with target groups and auto-scaling.
- **The Results:** Near-linear scaling achieved. Handled 100 users at **1,269 RPS** with 0% failures.
- **Target Policies:** 50% CPU target triggered earlier scaling than 70%, trading cost efficiency for spike-headroom.
- **Chaos Resilience:** Killed 2 tasks mid-test. ALB drained connections perfectly, ECS replaced tasks automatically.

Midterm Mastery (Part I): The Architecture of Tradeoffs

- **Imperative to Declarative:** Transitioning from manual EC2 configuration to repeatable, idempotent Terraform modules.
- **Statelessness:** HW2 proved independent in-memory state breaks distributed systems; externalizing state (like S3 for MapReduce) is mandatory.
- **Concurrency Primitives:** `sync.Map` is 3.4× faster for read-heavy loads, but `RWMutex` excels in standard GET/POST ratios.
- **Amdahl's Law in Practice:** 3 distributed mappers were faster than 1 sequential mapper.
- **The Rule:** Everything is a tradeoff.

Designing for Failure: Crash & Recover

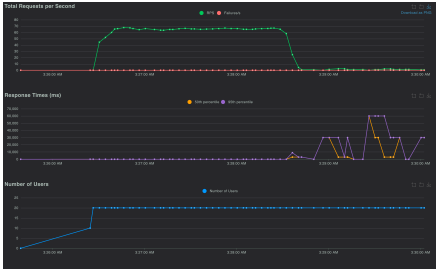
The Setup: Product Service enriches search results via a dependent Inventory Service.

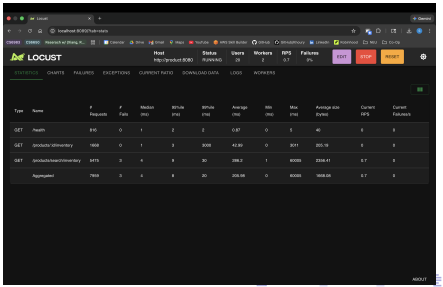
The Twist: Inventory Service has a `/chaos/:mode` endpoint to inject a 3-second latency delay.

Mode	Timeout	Circuit Breaker	Bulkhead	Behavior
`none`	30s	No	No	Cascading failure
`circuit-breaker`	500ms	5 fail $\rightarrow$ open	No	Graceful degradation
`bulkhead`	500ms	5 fail $\rightarrow$ open	10 slots	Bounded concurrency

Midterm Mastery (Part II): No Protection (Cascading Failure)

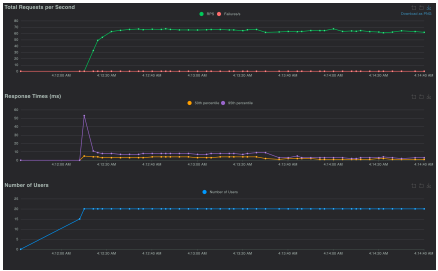
- **Condition:** Standard 30-second HTTP timeout.
- **Event:** 3-second delay injected.
- **Result:** Product Service threads block waiting for inventory.
- **Impact:** Throughput collapses by 99% (65.3 RPS $\rightarrow$ 0.7 RPS). Max latency hits 60 seconds.

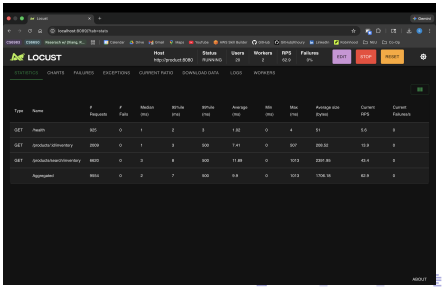




Midterm Mastery (Part II): Circuit Breaker (Fail-Fast)

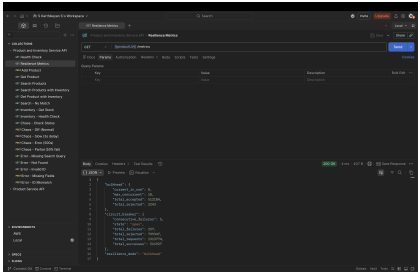
- **The Fix:** 500ms timeout + Circuit Breaker (Opens after 5 failures).
- **Event:** 3-second delay injected.
- **Result:** Circuit trips to OPEN. Service immediately returns degraded inventory data instead of hanging.
- **Impact:** Retained 96% of baseline throughput (62.9 RPS). Median latency drops to 2ms (skipping the network call entirely).

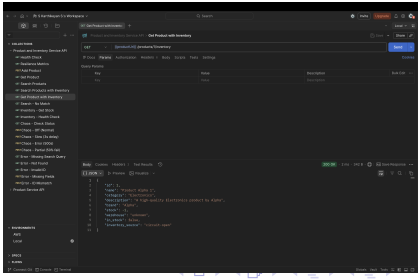




Midterm Mastery (Part II): Bulkhead Isolation

- **The Fix:** Circuit Breaker + Bulkhead Pattern (10 concurrent slots).
- **Event:** 50-user heavy load + 3-second delay injected.
- **Result:** 2,243 excess requests were instantly rejected (`bulkhead-rejected`) before the circuit even opened.
- **Impact:** Prevents thread exhaustion. System seamlessly recovered to ∼827 RPS during sustained chaos.





Conclusion:

- **Without Protection:** A single slow dependency causes total system failure.
- **With Protection:** We trade *data completeness* (live inventory) for *availability* (fast search results).
- **Client Awareness:** The API explicitly returns `inventory_source: "circuit-open"` so the client UI can react accordingly (e.g., hiding the "In Stock" badge).